MATLAB Essencial

Ferramentas para Análise e Simulação em Engenharia

XXI ENEEB

João Maria Machado

joaomaria99@ua.pt

7 de março

Objetivo do Workshop

- Explicar os conceitos e ferramentas básicas do **MATLAB**.
- Capacitar os participantes para a resolução de **problemas simples de simulação em engenharia**.

Agenda

- **Parte I — Fundamentos de MATLAB (40 min)**
  - Sintaxe e semelhanças com Python
  - Matrizes e operações
  - Indexação e criação funcional
  - Controlo de fluxo (if, while, for)
  - Funções e organização de scripts
  - Visualização e personalização de plots
- **Parte II — Caso Prático (20 min)**
  - Modelação de um problema farmacocinético
  - Resolução numérica de uma ODE (`ode45`)
  - Análise dos resultados e impacto do parâmetro k

MATLAB vs Python — Semelhanças Fundamentais

A sintaxe do MATLAB é muito semelhante à do Python, bastante simples, especialmente para quem já tem bases de programação.

Python

```python
x = 10
y = 2.5
z = x + y

v = [1, 2, 3, 4]

if x > 5:
    print("Maior que 5")
```

MATLAB

```mathematica
x = 10;
y = 2.5;
z = x + y;

v = [1 2 3 4];

if x > 5
    disp("Maior que 5")
end
```

Nota importante:

- Em MATLAB, as linhas terminam com `;` para **suprimir output**.

Matrizes — O Objeto Fundamental do MATLAB

Em MATLAB, **tudo é uma matriz** (escalares e vetores são casos particulares).

1) Escalar (1x1)

```
a = 5;
```

2) Vetor linha (1xn)

```
v = [1 2 3 4];
```

3) Vetor coluna (nx1)

```
v_col = [1; 2; 3; 4];
```

4) Matriz (mxn)

```
A = [1 2 3;
     4 5 6;
     7 8 9];
```

Operações com Matrizes

Operações básicas

```
A = [1+2i  3-1i;
     4     2+3i];

B = [2-i   1;
     0     5+2i];

C1 = A + B;
C2 = A - B;
C3 = A * B;
```

Operações elemento a elemento (case-wise)

```
C4 = A .* B;
C5 = A ./ B;
C6 = A .^ 2;
```

Funções e Constantes Reservadas em MATLAB

Funções matemáticas comuns

```
x = pi/4;

y1 = cos(x);
y2 = log(exp(2));
y3 = sqrt(16);
y4 = abs(-3+4i);
y5 = round(3.6);
```

Constantes e valores especiais

```
a = pi;
b = 2 + 3i;
c = eps;
d = realmin;
e = realmax;
```

Indexação de Matrizes

```
A = [1 2 3;
     4 5 6;
     7 8 9];

x   = A(1,2);       % elemento (linha,coluna)
row = A(2,:);       % linha completa
col = A(:,3);       % coluna completa

sub = A(1:2,2:3);   % submatriz

A(3,3) = 99;        % alterar elemento
A(:,1) = 0;         % alterar coluna inteira
```

- Indexação começa em 1
- : seleciona intervalos ou dimensões completas

Criação Funcional de Matrizes

```
Z = zeros(2,3);
O = ones(3,1);
I = eye(3);

R  = rand(2,2);
Ri = randi(10,2,3);


v = 0:0.1:1;        % vetor com passo definido
lin = linspace(0,1,5);
```

- Muito usado em simulação para gerar condições iniciais
- Evita construir matrizes manualmente

Controlo de Fluxo — if

```
x = 7;

if x > 10
    disp("Maior que 10")
elseif x > 5
    disp("Entre 6 e 10")
else
    disp("Menor ou igual a 5")
end
```

- Não existem parênteses obrigatórios
- Blocos terminam sempre com `end`

Controlo de Fluxo — while

```
n = 1;

while n < 5
    disp(n)
    n = n + 1;
end


% Exemplo típico em simulação
erro = 1;
while erro > 1e-3
    erro = erro / 2;
end
```

Controlo de Fluxo — for

```
A = [1 2 3;
     4 5 6];

for i = 1:size(A,1)
    for j = 1:size(A,2)
        disp(A(i,j))
    end
end


% Alternativa linear
for val = A(:)
    disp(val)
end
```

Criar e Usar Funções em MATLAB

Definição de uma função (ficheiro .m com o mesmo nome)

```
function y = quadrado(x)
    y = x^2;
end
```

Função com múltiplos inputs e outputs

```
function [soma, produto] = operacoes(a,b)
    soma = a + b;
    produto = a * b;
end
```

Utilização

```
r1 = quadrado(5);

[s, p] = operacoes(3,4);
```

Visualização de Dados — Sintaxe Básica de Plot

```
x = 0:0.1:2*pi;
y = sin(x);

plot(x, y)


plot(x, y, 'r--', 'LineWidth', 2)
xlabel('x')
ylabel('sin(x)')
title('Exemplo de Plot')
grid on
```

- `plot(x,y)` desenha y em função de x
- Argumentos adicionais controlam estilo (cor, linha, espessura)
- Funções auxiliares adicionam contexto ao gráfico

Exemplo — Subplots

```
x = 0:0.01:2*pi;

figure

subplot(2,2,1)
plot(x,sin(x)), title('sin(x)'), grid on

subplot(2,2,2)
plot(x,cos(x)), title('cos(x)'), grid on

subplot(2,2,3)
plot(x,exp(-0.2*x).*sin(2*x))
title('Damped'), grid on

subplot(2,2,4)
plot(x,sin(x).^2)
title('sin^2'), grid on
```

Figura gerada pelo código

Personalização de Plots

```
x = 0:0.1:10;
y = sin(x);

plot(x,y,'r--o','LineWidth',2,...
         'MarkerSize',6,...
         'MarkerFaceColor','r')
```

Cor e Forma

`'r--o'`

- Cor → 'r','g','b','k','m','c','y'
- Linha → '-' '–' ':' '-.'
- Marcadores → 'o','x','+','*','s','d','v','<','>'

Boas práticas

```
title('Título claro')
xlabel('Variável independente')
ylabel('Variável dependente')
legend('Descrição','Location','best')
grid on
```

Personalização de Plots

```
x = 0:0.1:10;
y1 = sin(x);
y2 = 0.5*cos(x);

figure
plot(x,y1,'b-','LineWidth',2); hold on
plot(x,y2,'r--o','LineWidth',1.5,...
     'MarkerSize',5,...
     'MarkerFaceColor','r');
hold off

title('Exemplo de Customização')
xlabel('x')
ylabel('Amplitude')
legend('sin(x)','0.5cos(x)',...
       'Location','best')
grid on
```

Figura gerada pelo código (plot com cores, marcadores e legenda)

Boas Práticas num Script MATLAB

```
%% Limpeza inicial
clear;
close all;
clc;

%% Parâmetros
T = 10;
dt = 0.01;
t = 0:dt:T;

%% Modelo
y = exp(-0.3*t).*sin(2*pi*t);

%% Visualização
figure
plot(t,y,'LineWidth',2)
title('Simulação do sistema')
xlabel('Tempo (s)')
ylabel('Resposta')
grid on
```

- Usar %% para criar secções
- Comentar variáveis importantes
- Organizar o script por blocos
- Separar:
  - Parâmetros
  - Modelo
  - Visualização

Caso Prático — Modelo Farmacocinético

Problema:
Modelar a concentração de um fármaco no sangue após administração intravenosa.

Hipótese: Modelo monocompartimental
A variação da concentração é proporcional à quantidade presente:

$$\frac{dC(t)}{dt} = -k\,C(t)$$

- $C(t)$ — concentração do fármaco
- k — constante de eliminação
- Condição inicial: $C(0) = C_0$

Objetivo da simulação:
- Resolver numericamente a ODE
- Visualizar a evolução temporal da concentração
- Analisar o efeito de diferentes valores de k

Modelo Farmacocinético — Condições Iniciais

Consideremos:

- Dose intravenosa inicial: $C_0 = 10$ mg/L
- Constante de eliminação: $k = 0.4$ h^{-1}
- Intervalo de simulação: $t \in [0, 24]$ horas

Tradução para MATLAB

```
C0 = 10;      % concentração inicial (mg/L)
k  = 0.4;     % constante de eliminação (1/h)

tspan = [0 24];   % intervalo temporal
```

Objetivo: resolver numericamente

$$\frac{dC}{dt} = -kC$$

com $C(0) = C_0$.

Resolver ODEs com ode45

O que faz o ode45?

Resolve numericamente equações diferenciais da forma:

$$\frac{dy}{dt} = f(t, y)$$

Passo 1 — Criar função do modelo (ficheiro: modelo.m)

```
function dCdt = modelo(t,C)

k = 0.4;
dCdt = -k*C;

end
```

Passo 2 — Chamar o ode45 no script principal

```
C0 = 10;
tspan = [0 24];

[t,C] = ode45(@modelo, tspan, C0);
```

Força!

Agora é convosco.

Implementem a simulação do modelo farmacocinético em MATLAB:

$$\frac{dC}{dt} = -kC, \qquad C(0) = C_0$$

- Definir C_0, k e `tspan`
- Implementar a ODE e resolver com `ode45`
- Fazer um plot de $C(t)$ com título, eixos e grelha

Resultados Esperados

Placeholder Figura 1
C(t) vs tempo (ode45)

Placeholder Figura 2
Comparação de diferentes valores de k

- $C(t)$ deve decair exponencialmente ao longo do tempo
- Valores maiores de $k \Rightarrow$ eliminação mais rápida (decai mais depressa)

Obrigado pela atenção!

Dúvidas?

João Maria Machado

joaomaria99@ua.pt